

Práctica 05 — El movimiento en diagonal

Programación de Videojuegos · SMR · FP Lumara Duración: 1 sesión · Práctica previa: 04

Lo que aprendes hoy: qué es un vector de dirección y por qué hay que normalizarlo.

1. Empieza por encontrar el fallo

Abre el proyecto de partida y ejecútalo con **F6**. Es el resultado de la práctica 04, ya montado.

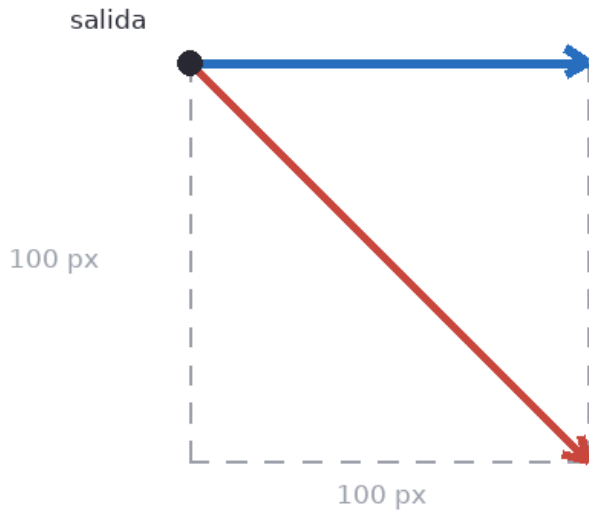
Ahora haz esta prueba, **cronometrando mentalmente**:

1. Coloca el personaje en la esquina superior izquierda.
2. Cruza la pantalla pulsando solo la flecha derecha. Cuenta lo que tarda.
3. Vuelve a la esquina y cruza pulsando **derecha y abajo a la vez**, en diagonal.
4. Cuando toques el borde inferior pulsa **derecha y arriba a la vez**.

¿Has visto que tarda lo mismo en línea recta que dando toda la vuelta? Cuando el personaje se mueve en diagonal avanza un **41 % más rápido**. Y eso, en un juego, es un fallo, porque los jugadores lo descubren enseguida y acaban moviéndose siempre en diagonal.

[CAPTURA 05.1]

Un segundo de movimiento a 100 px/s



solo derecha
recorre 100 px

derecha y abajo
recorre 141 px

141 / 100 = 1,41
un 41 % más de camino
en el mismo tiempo

$$\sqrt{(100^2 + 100^2)} = \sqrt{20000} \approx 141,4$$

"Pulsando dos teclas, el personaje recorre más distancia en el mismo tiempo."

2. Por qué pasa

Mira el código de la práctica 04. Si pulsas **derecha** y **abajo** a la vez, se ejecutan **dos líneas**:

```
position.x += velocidad * delta    # avanza 100 px/s hacia la derecha
position.y += velocidad * delta    # avanza 100 px/s hacia abajo
```

El personaje avanza 100 hacia la derecha **y además** 100 hacia abajo. Pero la distancia que recorre de verdad no es 100, es la diagonal de un cuadrado de lado 100.

Y esa diagonal, por el teorema de Pitágoras, mide:

$$\sqrt{(100^2 + 100^2)} = \sqrt{20000} \approx 141,4$$

De ahí sale el 41 % de más.

3. La solución: vectores

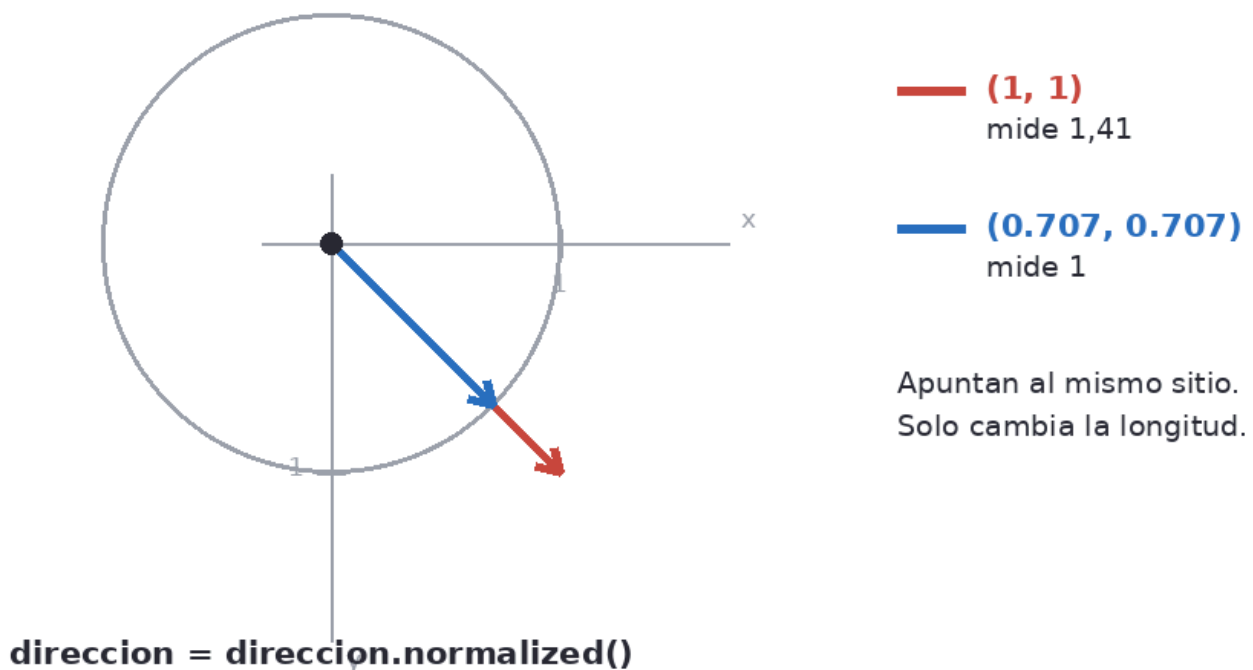
Un **vector** es un par de números que indica una dirección: cuánto en **x** y cuánto en **y**. En Godot se escribe **Vector2(x, y)**.

Vector	Significa
Vector2(1, 0)	Todo a la derecha
Vector2(-1, 0)	Todo a la izquierda
Vector2(0, -1)	Todo hacia arriba
Vector2(1, 1)	Derecha y abajo... pero mide 1,41, no 1

Normalizar un vector es dejarlo apuntando a la misma dirección pero con longitud exactamente 1. Así, vayas donde vayas, la distancia recorrida es siempre la misma.

[CAPTURA 05.2]

Normalizar: la misma dirección, con longitud 1



“Normalizar conserva la dirección y arregla la longitud.”

4. Paso a paso

4.1 Reescribir el movimiento con un vector

1. Abre `scripts/personaje.gd`.
2. Sustituye todo el contenido por esto:

```
extends Sprite2D

var velocidad := 100.0

func _process(delta: float) -> void:
    var direccion := Vector2.ZERO

    if Input.is_action_pressed("mover_izquierda"):
        direccion.x -= 1
    if Input.is_action_pressed("mover_derecha"):
        direccion.x += 1
    if Input.is_action_pressed("mover_arriba"):
        direccion.y -= 1
    if Input.is_action_pressed("mover_abajo"):
        direccion.y += 1

    direccion = direccion.normalized()
    position += direccion * velocidad * delta
```

3. Ejecuta y repite la prueba del cronómetro. Ahora la diagonal tarda lo mismo que la horizontal.

4.2 Ahora la versión corta

Lo que acabas de escribir (leer cuatro teclas, montar un vector y normalizarlo) es tan habitual que Godot lo trae ya hecho en una sola función.

4. Sustituye **todo el cuerpo de la función** por estas dos líneas:

```
extends Sprite2D

var velocidad := 100.0

func _process(delta: float) -> void:
    var direccion := Input.get_vector(
        "mover_izquierda", "mover_derecha", "mover_arriba", "mover_abajo")
    position += direccion * velocidad * delta
```

5. Ejecuta otra vez y comprueba que se comporta **exactamente igual**.

6. Guarda la escena como `escenas/personaje.tscn` si no lo estaba ya.

El orden de los cuatro nombres dentro de `get_vector` importa mucho: primero izquierda, luego derecha, luego arriba y por último abajo. Si los cambias de sitio, el personaje se moverá al revés.

5. El código, línea a línea

`var direccion := Vector2.ZERO` `Vector2.ZERO` es el vector `(0, 0)`, es decir, "quieto". Empezamos ahí en cada fotograma y vamos sumando según las teclas.

Fíjate en que esta variable está **dentro** de la función: nace y muere en cada fotograma. `velocidad`, en cambio, está fuera y dura siempre.

`direccion.x -= 1` No se suma la velocidad, se suma **un 1**. La dirección solo dice *hacia dónde*, no *cuánto*. La velocidad se aplica al final, una sola vez.

`direccion = direccion.normalized()` Devuelve el mismo vector con longitud 1. Con `(1, 1)` devuelve aproximadamente `(0.707, 0.707)`. Con `(0, 0)` devuelve `(0, 0)`, así que estar quieto sigue funcionando.

`position += direccion * velocidad * delta` Se lee de derecha a izquierda: la dirección, multiplicada por la velocidad, multiplicada por el tiempo transcurrido. Multiplicar un vector por un número multiplica sus dos componentes a la vez, así que esta única línea sustituye a las cuatro de la práctica 04.

`Input.get_vector(...)` Hace exactamente lo mismo que las diez líneas anteriores: lee las cuatro acciones, monta el vector y lo normaliza. Se usará en el resto del curso.

6. Pruébalo

- ☐ La diagonal tarda lo mismo que la horizontal en cruzar la pantalla.
- ☐ El personaje se sigue moviendo en las ocho direcciones.
- ☐ Al soltar todas las teclas se para del todo.
- ☐ El código final tiene dos líneas dentro de la función, no diez.

7. La teoría en cinco líneas

- Un **vector** guarda una dirección con dos números, **x** e **y**.
- **Normalizar** deja el vector apuntando igual pero con longitud 1.
- Separar *dirección* de *velocidad* permite cambiar una sin tocar la otra.
- Multiplicar un vector por un número afecta a sus dos componentes de golpe.
- **Input.get_vector()** lee cuatro acciones y devuelve el vector ya normalizado.

8. Si no te funciona

Lo que ves	Qué pasa	Cómo se arregla
Sigue yendo más rápido en diagonal	Falta la línea de normalized()	Compruébala, o pasa a get_vector
El personaje no se para nunca	No reinicias direccion a ZERO cada fotograma	Esa línea va dentro de la función
Se mueve al revés	El orden de los nombres en get_vector está cambiado	Izquierda, derecha, arriba, abajo
Invalid call to 'normalized'	Has puesto normalized sin los paréntesis	Es una función: normalized()
Va lentísimo	Has normalizado después de multiplicar por la velocidad	Primero se normaliza, luego se multiplica
No se mueve	El nombre de una acción está mal escrito	Cópialo del Mapa de Entrada

9. Retos

Verde · Añade un `print(direccion)` dentro de la función y mira el panel **Salida** mientras mueves el personaje. Comprueba con tus ojos que en diagonal aparecen los `0.707` . Quítalo después: escribir en pantalla 60 veces por segundo ralentiza el juego.

Amarillo · Haz que el personaje **no pueda salirse de la pantalla**. Pista: la pantalla mide 320 x 180, y existe una función llamada `clamp()` que obliga a un número a quedarse entre un mínimo y un máximo.

Rojo · Que el personaje no arranque ni frene de golpe, sino que acelere y desacelere de forma suave. Pista: guarda la velocidad actual en una variable del nodo y acércala poco a poco a la deseada con `lerp()` o con `move_toward()` . Busca en la documentación qué hace cada una y elige.

10. Autoevaluación

- ☐ Sé explicar por qué sin normalizar se va más rápido en diagonal.
- ☐ Sé qué es un `Vector2` y qué guarda.
- ☐ Sé qué hace `normalized()` .
- ☐ Entiendo por qué la dirección se construye con unos y no con la velocidad.
- ☐ Sé usar `Input.get_vector()` y en qué orden van sus cuatro nombres.
- ☐ Sé la diferencia entre una variable declarada dentro y fuera de una función.

11. Vocabulario

Español	Inglés	Qué es
Vector	Vector	Par de números que indica dirección
Normalizar	Normalize	Dejar el vector con longitud 1
Longitud, módulo	Length, magnitude	Lo que mide el vector
Variable local	Local variable	La que vive solo dentro de una función

12. Entrega

Comprime la carpeta como `apellido_practica05.zip` y súbela al Aula Virtual de Alexia.

Capturas necesarias en esta práctica

Código	Contenido
05.1	Esquema de las dos flechas, 100 px frente a 141 px
05.2	Esquema del vector (1,1) antes y después de normalizar